Name: Daljeet Singh
Student ID: 3169763

PART - I

```
pi@3169763Pi: ~/Workspace/
pi@3169763Pi:~/Workspace/assign03 $ ./assign03_red
Error: no filename passed
pi@3169763Pi:~/Workspace/assign03 $ ./assign03_red output.txt
pi@3169763Pi:~/Workspace/assign03 $ cat ./output.txt
Hello Daljeet Singh, I'm being redirected to a file instead of the I/O display device!
pi@3169763Pi:~/Workspace/assign03 $ ./assign03_red ouput.txt
pi@3169763Pi:~/Workspace/assign03 $ cat ./output.txt
Hello Daljeet Singh, I'm being redirected to a file instead of the I/O display device!
pi@3169763Pi:~/Workspace/assign03 $ ./assign03_red new_output.txt
pi@3169763Pi:~/Workspace/assign03 $ cat ./new_output.txt
Hello Daljeet Singh, I'm being redirected to a file instead of the I/O display device!
pi@3169763Pi:~/Workspace/assign03 $ ls al new_output.txt
ls: cannot access 'al': No such file or directory
new_output.txt
pi@3169763Pi:~/Workspace/assign03 $ ls -al new_output.txt
-rw-r--r-- 1 pi pi 87 Mar 30 13:01 new_output.txt
pi@3169763Pi:~/Workspace/assign03 $
```

1. What dup2( ) helped to achieve:

Ans: The dup2( ) system call allowed me to redirect the process's standard output to a specific file. By duplicating the file descriptor of the opened file onto the standard output descriptor. When the program uses printf(), the message is written into the file rather than displayed on the I/O devices.

2. Benefits of the file descriptor duplication:

Ans: Duplicating file descriptors with dup2( ) provides a powerful way to manage I/O without modifying the source code. They allow a program to reuse standard operations like printf() and scanf() with files, pipes or devices without changing the rest of the code. This is very useful in shell commands and communication between processes.

PART – II

Question 2:

Sometimes a consumer message appear before the matching producer message this occurs because the printf statements are located outside the critical section. After the producer puts the items in the buffer and release the semaphores, the schedular run a consumer immediately and the consumer can eat that item before the producer's printf happens.

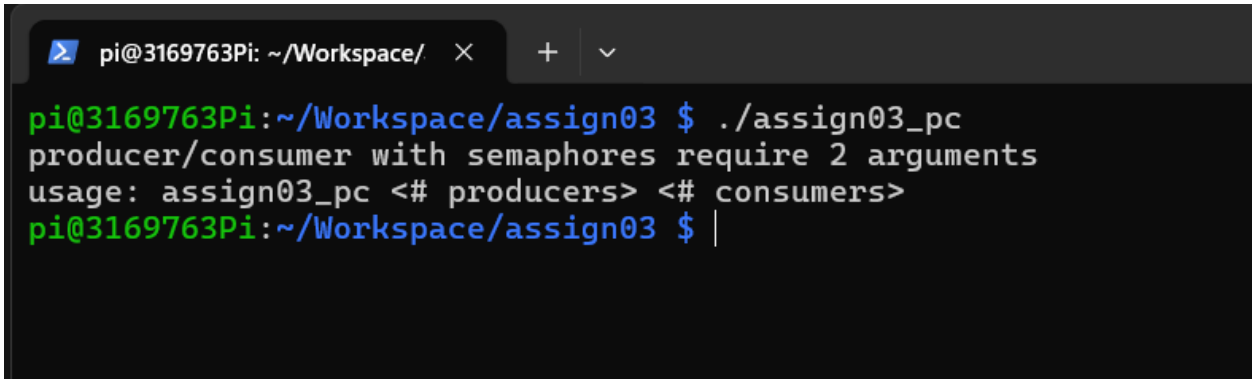
Question 3:

In the 0 producer and 3 consumer case, no real data will ever produced. In this program, the producer finish immediately and the main thread send items to the consumers and consumer will wait indefinitely for the data.

Question 4:

In the 4 producers and 0 consumers case, producers produces data and stored in the buffer until the buffer becomes full. Once the buffer is full, any producer attempts to add more data will be blocked. Since no consumers exist to remove items and call sem_post(&empty), the producers will remain blocked forever once the buffer is full.

Question 5:



```
pi@3169763Pi: ~/Workspace/

pi@3169763Pi:~/Workspace/assign03 $ ./assign03_pc 1
producer/consumer with semaphores require 2 arguments
usage: assign03_pc <# producers> <# consumers>
pi@3169763Pi:~/Workspace/assign03 $
```

```
pi@3169763Pi: ~/Workspace/

pi@3169763Pi:~/Workspace/assign03 $ ./assign03_pc 1 2
producer 0 generates data 0-0
consumer 0 eats data = 0-0
producer 0 generates data 0-1
consumer 1 eats data = 0-1
producer 0 generates data 0-2
consumer 0 eats data = 0-2
producer 0 generates data 0-3
consumer 1 eats data = 0-3
producer 0 generates data 0-4
consumer 0 eats data = 0-4
Producers ended!
pi@3169763Pi:~/Workspace/assign03 $
```

```
pi@3169763Pi: ~/Workspace/

pi@3169763Pi:~/Workspace/assign03 $ ./assign03_pc 3 1
producer 1 generates data 1-0
producer 0 generates data 0-0
producer 2 generates data 2-0
consumer 0 eats data = 1-0
consumer 0 eats data = 0-0
consumer 0 eats data = 2-0
producer 1 generates data 1-1
consumer 0 eats data = 1-1
producer 0 generates data 0-1
producer 2 generates data 2-1
consumer 0 eats data = 0-1
consumer 0 eats data = 2-1
producer 1 generates data 1-2
consumer 0 eats data = 1-2
producer 0 generates data 0-2
consumer 0 eats data = 0-2
consumer 0 eats data = 2-2
producer 2 generates data 2-2
producer 0 generates data 0-3
consumer 0 eats data = 0-3
producer 1 generates data 1-3
consumer 0 eats data = 1-3
producer 2 generates data 2-3
consumer 0 eats data = 2-3
producer 1 generates data 1-4
producer 0 generates data 0-4
consumer 0 eats data = 1-4
consumer 0 eats data = 0-4
producer 2 generates data 2-4
consumer 0 eats data = 2-4
Producers ended!
pi@3169763Pi:~/Workspace/assign03 $
```

```
pi@3169763Pi: ~/Workspace/

pi@3169763Pi:~/Workspace/assign03 $ ./assign03_pc 3 4
producer 0 generates data 0-0
producer 1 generates data 1-0
consumer 1 eats data = 0-0
consumer 0 eats data = 1-0
producer 2 generates data 2-0
consumer 2 eats data = 2-0
producer 0 generates data 0-1
producer 1 generates data 1-1
consumer 3 eats data = 0-1
consumer 1 eats data = 1-1
producer 2 generates data 2-1
consumer 0 eats data = 2-1
producer 1 generates data 1-2
producer 0 generates data 0-2
consumer 3 eats data = 1-2
consumer 2 eats data = 0-2
producer 2 generates data 2-2
consumer 1 eats data = 2-2
producer 0 generates data 0-3
consumer 0 eats data = 0-3
producer 1 generates data 1-3
consumer 3 eats data = 1-3
producer 2 generates data 2-3
consumer 2 eats data = 2-3
producer 0 generates data 0-4
consumer 1 eats data = 0-4
producer 1 generates data 1-4
consumer 0 eats data = 1-4
producer 2 generates data 2-4
consumer 3 eats data = 2-4
Producers ended!
pi@3169763Pi:~/Workspace/assign03 $
```

```
pi@3169763Pi: ~/Workspace/
pi@3169763Pi:~/Workspace/assign03 $ ./assign03_pc 0 3
0 producers, 3 consumers: Consumers will wait indefinitely for data.
pi@3169763Pi:~/Workspace/assign03 $
```

```
pi@3169763Pi: ~/Workspace/
pi@3169763Pi:~/Workspace/assign03 $ ./assign03_pc 4 0
4 producer, 0 consumers: data can be produced, but nobody eat it, so the program
 would block once the buffer becomes full.
pi@3169763Pi:~/Workspace/assign03 $
```

pi@3169763Pi: ~/Workspace/

```
pi@3169763Pi:~/Workspace/assign03 $ ./assign03_pc 5 5
producer 1 generates data 1-0
producer 3 generates data 3-0
producer 0 generates data 0-0
consumer 2 eats data = 0-0
producer 2 generates data 2-0
consumer 1 eats data = 2-0
producer 4 generates data 4-0
consumer 3 eats data = 3-0
consumer 3 eats data = 4-0
consumer 0 eats data = 1-0
producer 3 generates data 3-1
producer 0 generates data 0-1
producer 2 generates data 2-1
producer 4 generates data 4-1
consumer 2 eats data = 0-1
consumer 4 eats data = 2-1
producer 1 generates data 1-1
consumer 1 eats data = 3-1
consumer 4 eats data = 1-1
consumer 3 eats data = 4-1
producer 0 generates data 0-2
producer 3 generates data 3-2
consumer 2 eats data = 0-2
producer 2 generates data 2-2
consumer 2 eats data = 2-2
producer 1 generates data 1-2
consumer 0 eats data = 3-2
consumer 1 eats data = 4-2
consumer 2 eats data = 1-2
producer 4 generates data 4-2
producer 3 generates data 3-3
producer 0 generates data 0-3
consumer 4 eats data = 3-3
consumer 4 eats data = 2-3
producer 2 generates data 2-3
consumer 3 eats data = 0-3
producer 1 generates data 1-3
consumer 1 eats data = 1-3
producer 4 generates data 4-3
consumer 2 eats data = 4-3
consumer 4 eats data = 0-4
producer 0 generates data 0-4
producer 2 generates data 2-4
consumer 0 eats data = 2-4
producer 1 generates data 1-4
consumer 3 eats data = 1-4
producer 4 generates data 4-4
consumer 1 eats data = 4-4
consumer 1 eats data = 3-4
producer 3 generates data 3-4
Producers ended!
pi@3169763Pi:~/Workspace/assign03 $
```